



Hafta 4 — Sipariş Motoru, Admin Siparişler, SignalR, UX, CI/CD, Garson Çağır, Fiş & Saat Dilimi Fix

Proje: QR Menü Otomasyonu | Tarih: 8–11 Mart 2026 | Harcanan Süre: ~42+ Saat

1. Proje Dosya Yapısı (Hafta 4 Güncellemeleri)

```
QRMenu/
├── src/
│   ├── QRMenu.Core/
│   │   ├── Entities/
│   │   │   └── Siparis.cs          ← RowVersion default düzeltmesi (Array.Empty)
│   │   ├── Enums/
│   │   │   └── SiparisDurum.cs    ← 9-state enum (Sepette..İade)
│   │   ├── Interfaces/
│   │   │   └── ISiparisService.cs ← [YENİ] Sipariş iş kurallarının kontratı
│   │   └── QRMenu.Data/
│   │       └── Services/
│   │           └── SiparisService.cs ← [YENİ] State Machine + Transaction + Concurrency
│   └── QRMenu.Web/
│       ├── Program.cs             ← DI: ISiparisService + AddSignalR + MapHub
│       ├── Hubs/
│       │   └── MenuHub.cs          ← [YENİ] SignalR Hub – gerçek zamanlı menü guncellemeleri
│       ├── Controllers/
│       │   ├── SiparisController.cs ← [YENİ] 5 endpoint + SignalR SiparisGuncellendi broadcast
│       │   ├── AdminController.cs   ← [+19 Endpoint] Kategori/Ürün/Opsiyon CRUD + Sipariş Yönetim
│       │   ├── MenuController.cs    ← [+1 Endpoint] /menu/json – SignalR güncellemesi için JSON
│       │   └── SepetController.cs    ← Index() → redirect (ayrı sayfa kaldırıldı)
│       ├── Middleware/
│       │   └── TokenValidationMiddleware.cs ← AJAX-aware 401 JSON + /hubs exclude
│       ├── ViewModels/
│       │   └── UrunYonetimViewModel.cs ← [YENİ] Kategori + Ürün + Opsiyon form modelleri
│       ├── Views/
│       │   ├── Admin/
│       │   │   ├── Urunler.cshtml   ← [YENİ] Tam CRUD sayfası (3 modal + JS)
│       │   │   └── Siparisler.cshtml ← [YENİ] Sipariş yönetimi (stat kartlar + filtre + durum g
│       │   ├── Menu/
│       │   │   └── Index.cshtml      ← SignalR client + Bootstrap modal/toast + görsel render
│       │   └── Shared/
│       │       └── _AdminLayout.cshtml ← Sidebar: Ürünler + Siparişler linki aktif
│       └── wwwroot/
│           ├── css/
│           │   ├── admin-urunler.css ← [YENİ] Ürün yönetim sayfası stili
│           │   ├── admin-siparisler.css ← [YENİ] Sipariş yönetimi + Progress Stepper stili
│           │   └── menu.css           ← .product-ımg stili eklendi
│           ├── uploads/
│           │   └── urunler/          ← [YENİ] Ürün fotoğrafları dizini
│       └── .github/
```

```
|   └─ workflows/
|       └─ deploy.yml          ← [YENİ] GitHub Actions CI/CD pipeline (build → test → deploy)
|
|   └─ tests/
|       └─ QRMenu.Tests/
|           └─ SiparisServiceTests.cs ← [YENİ] +28 unit test (Toplam: 59)
```

1.2 — 9 Mart Dosya Değişiklikleri

```
src/QRMenu.Web/
├─ Controllers/
│   ├─ AdminController.cs    ← UrunSil() FK temizleme + opsiyon sıralama
│   └─ MenuController.cs    ← opsiyon OrderBy(EkFiyat) eklendi
├─ Views/Menu/
│   └─ Index.cshtml         ← opsiyon sıralama + sos grubu isMulti düzeltmesi
└─ wwwroot/css/
    ├─ admin-urunler.css    ← overflow: hidden → overflow-x: auto (mobil fix)
    └─ admin-siparisler.css ← overflow-x: auto + media query konsolidasyonu (mobil fix)
```

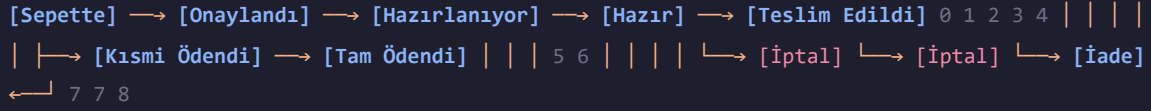
2. Planlanan vs. Gerçekleşen (Haftalık Hedefler)

Madde	Durum	Açıklama
Sipariş Motoru (State Machine)	✓ Tamamlandı	9 durumlu state machine ile sipariş yaşam döngüsü yönetimi.
Transaction ile Atomik Sipariş	✓ Tamamlandı	Sepet → Sipariş dönüşümü tek transaction içinde. Provider-aware tasarım.
RowVersion Concurrency Kontrolü	✓ Tamamlandı	Eş zamanlı durum güncellemelerinde çakışma tespiti ve kullanıcı bildirimi.
Sipariş Unit Testleri	✓ Tamamlandı	28 unit test: oluşturma, state geçişleri, iptal, sorgulama senaryoları.
Admin Kategori / Ürün / Opsiyon CRUD	✓ Tamamlandı	15 endpoint — Kategori chip UI, Ürün tablosu, Opsiyon modal, fotoğraf upload.
Admin Siparişler Sayfası	🚀 Bonus	5 stat kartı + durum/masa filtresi + state machine butonları + detay modal.
Sipariş Gönder — Bootstrap Modal + Toast	🚀 Bonus	Browser confirm/alert yerine profesyonel onay modali + toast bildirimi.
Ayrı Sepet Sayfası Kaldırıldı	🚀 Bonus	/sepet → redirect. Offcanvas sepet tek arayüz olarak kullanılıyor.
Middleware AJAX-Aware 401	🚀 Bonus	AJAX isteklerinde JSON {"success":false}, tarayıcıda düz metin 401 yanıtı.
Menü Görsel Render Düzeltmesi	🚀 Bonus	Admin'den yüklenen görseller menüde gösterilmeyordu — koşullu eklendi.

Madde	Durum	Açıklama
SignalR Gerçek Zamanlı Menü	 Bonus	Admin değişiklikleri anlık menüye yansır. Hub + 11 notify noktası + JS client.
SignalR Sipariş Bildirimi	 Bonus	Yeni sipariş ve durum değişikliklerinde admin paneline anlık bildirim + Web Audio ses.
Progress Stepper UI	 Bonus	Stat kartları yerine yatay stepper bar — daire + çizgi ile sipariş durumu görselleştirme.
GitHub Actions CI/CD	 Bonus	Otomatik build → test → deploy pipeline. Secrets ile güvenli bağlantı bilgisi enjeksiyonu.

3. Sipariş Motoru — State Machine Mimarisi

3.1 Durum Akış Diyagramı



3.2 State Machine Kural Tablosu

Mevcut Durum	Enum	İzin Verilen Geçişler
Sepette	0	Onaylandı
Onaylandı	1	Hazırlanıyor, İptal
Hazırlanıyor	2	Hazır, İptal
Hazır	3	Teslim Edildi
Teslim Edildi	4	Kısmi Ödendi, Tam Ödendi, İade
Kısmi Ödendi	5	Tam Ödendi
Tam Ödendi	6	İade
İptal	7	— Son durum (terminal) —
İade	8	— Son durum (terminal) —

3.3 State Machine Kod Implementasyonu

```

private static readonly Dictionary<SiparisDurum, SiparisDurum[]> _gecisKurallari = new()
{
    [SiparisDurum.Sepette] = new[] { SiparisDurum.Onaylandi },
    [SiparisDurum.Onaylandi] = new[] { SiparisDurum.Hazirlaniyor, SiparisDurum.Iptal },
    [SiparisDurum.Hazirlaniyor] = new[] { SiparisDurum.Hazir, SiparisDurum.Iptal },
    [SiparisDurum.Hazir] = new[] { SiparisDurum.TeslimEdildi },
    [SiparisDurum.TeslimEdildi] = new[] { SiparisDurum.KismiOdendi, SiparisDurum.TamOdendi, SiparisDurum.Iade }
}

```

```
[SiparisDurum.KismiOdendi] = new[] { SiparisDurum.TamOdendi },
[SiparisDurum.TamOdendi] = new[] { SiparisDurum.Iade },
[SiparisDurum.Iptal] = Array.Empty<SiparisDurum>(),
[SiparisDurum.Iade] = Array.Empty<SiparisDurum>()
};
```

4. Transaction Yönetimi & Concurrency Kontrolü

Atomik Transaction

- Sepet → Sipariş dönüşümü tek transaction içinde
- SepetDetay → SiparisDetay kopyalama
- Sepet temizleme (ürünler + toplam sıfırlama)
- Hata durumunda otomatik rollback

Concurrency (RowVersion)

- Sipariş entity'sinde `byte[] RowVersion`
- EF Core `[Timestamp]` ile otomatik kontrol
- `DbUpdateConcurrencyException` yakalama
- Kullanıcıya "sayfa yenile" mesajı

4.1 Provider-Aware Transaction Stratejisi

Sorun: InMemory database provider'ı transaction desteklemiyor.

Çözüm: `ProviderName` kontrolü ile runtime'da provider tespiti. PostgreSQL (Npgsql) için

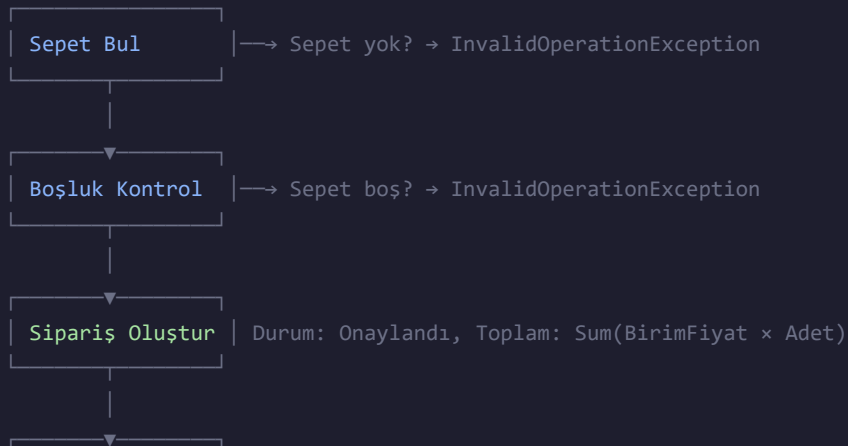
`ExecutionStrategy` + `BeginTransactionAsync` kullanılırken, InMemory testlerde transaction atlanıyor.

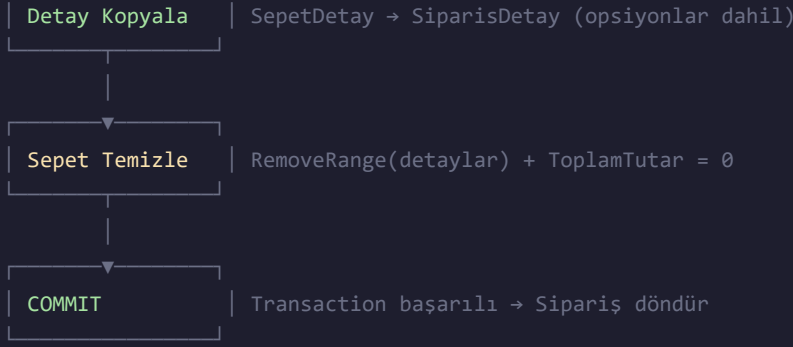
Bu şekilde aynı servis kodu hem production hem test ortamında sorunsuz çalışıyor.

```
var supportsTransactions = _context.Database.ProviderName
    != "Microsoft.EntityFrameworkCore.InMemory";

if (supportsTransactions)
{
    var strategy = _context.Database.CreateExecutionStrategy();
    return await strategy.ExecuteAsync(
        () => ExecuteSiparisOlusturAsync(sepetId, notlar, true));
}
return await ExecuteSiparisOlusturAsync(sepetId, notlar, false);
```

4.2 Sipariş Oluşturma Akış Şeması





5. Sipariş Controller — API Endpoint'leri

HTTP	Route	Açıklama
POST	/siparis/olustur	Sepetteki ürünleri siparişe çevirir. Body: { SepetId, Notlar? }
GET	/siparis/{id}	Sipariş detayını getirir (detay satırları + ürün bilgisi dahil).
POST	/siparis/durum-guncelle	State Machine kurallarına uygun durum geçişi. Body: { SiparisId, YeniDurum }
GET	/siparis/siparislerim	Aktif oturumun tüm siparişlerini listeler.
POST	/siparis/iptal/{id}	Siparişi iptal eder (sadece Onaylandı/Hazırlanıyor durumlarından).

6. Admin Panel — Ürün & Kategori & Opsiyon Yönetimi

6.1 Admin Controller Yeni Endpoint'ler (19 adet)

HTTP	Route	Açıklama
📁 Kategori Yönetimi		
GET	/admin/urunler	Ürün yönetim sayfası (View)
GET	/admin/kategoriler	Tüm kategorileri JSON olarak döndürür
POST	/admin/kategori-ekle	Yeni kategori oluşturur
POST	/admin/kategori-guncelle/{id}	Kategori bilgilerini günceller
POST	/admin/kategori-sil/{id}	Kategori siler (ürünü varsa engeller)
📁 Ürün Yönetimi		
GET	/admin/urun-detay/{id}	Ürün detayı (opsiyonlar dahil) JSON
POST	/admin/urun-ekle	Yeni ürün ekler (FromForm — fotoğraf ile)
POST	/admin/urun-guncelle/{id}	Ürün bilgilerini günceller (fotoğraf değişimi dahil)
POST	/admin/urun-sil/{id}	Ürünü ve fotoğrafını siler

HTTP	Route	Açıklama
POST	/admin/urun-toggle/{id}	Ürün aktif/pasif durumunu değiştirir
⚙️ Opsiyon Yönetimi		
POST	/admin/opsiyon-ekle	Ürüne opsiyon ekler (yeni oluşturur veya mevcut bağlar)
POST	/admin/opsiyon-sil/{id}	Ürün-opsiyon bağlantısını kaldırır
📋 Sipariş Yönetimi (4 Endpoint)		
GET	/admin/siparisler	Sipariş yönetim sayfası (View — stat kartlar + filtre + tablo)
GET	/admin/siparis-detay/{id}	Sipariş detayı JSON (ürünler + opsiyonlar + tutar)
POST	/admin/siparis-durum/{id}	State Machine kurallarına uygun sipariş durum güncelleme
GET	/admin/siparisler-json	Durum ve masa filtrelemeli sipariş listesi JSON

6.2 Fotoğraf Upload Güvenliği

Güvenlik Kontrolleri:

- **Dosya boyutu:** Maksimum 2MB (`MaxFileSize = 2 * 1024 * 1024`)
- **Uzanti kontrolü:** Sadece `.jpg` , `.jpeg` , `.png` , `.webp` kabul edilir
- **GUID dosya adı:** Orijinal dosya adı kullanılmaz — `Guid.NewGuid()` + `ext`
- **Eski dosya temizleme:** Fotoğraf güncellendiğinde veya ürün silindiğinde eski dosya disk'ten silinir
- **Depolama:** `/wwwroot/uploads/urunler/` dizinine kaydedilir

6.3 Admin UI Bileşenleri

📁 Kategori Yönetimi

- Chip tabanlı kategori listesi
- Her chip'te düzenle/sil ikonları
- Yeni kategori + inline düzenleme
- Ürün bağımlılık kontrolü (silme engeli)

📊 Ürün Tablosu

- Fotoğraf + ad + kategori + fiyat kolonları
- Aktif/pasif toggle switch
- Kategori dropdown filtresi
- İsme göre arama

📷 Fotoğraf Upload

- Dosya seçimi ile anlık önizleme
- FormData ile AJAX gönderim
- 2MB ve format kısıtlaması
- Güncelleme sırasında eski dosya silme

⚙️ Opsiyon Yönetimi

- Ürün bazlı modal ile opsiyon listesi
- Ad + Grup + Ek Fiyat ile yeni opsiyon
- Tek tıkla opsiyon kaldırma
- Mükerrer kontrolü

7. Admin Sipariş Yönetimi Sayfası

7.1 Sipariş Sayfası Bileşenleri

Progress Stepper Bar

- 5 adımlı yatay stepper (daire + bağlantı çizgisi)
- Onaylandı → Hazırlanıyor → Hazır → Teslim Edildi → Ödendi
- Sipariş satırına tıklayınca ilgili adıma kadar yeşil dolum
- İptal durumu: kırmızı vurgu
- Mini stat pill'leri ile durum sayıları

Filtre ve Arama

- Durum dropdown filtresi (9 durum)
- Masa numarası filtresi
- Temizle butonu
- Filtreleme AJAX ile anlık çalışır

Sipariş Tablosu

- ID, Masa, Durum, Toplam, Tarih kolonları
- Durum renk rozeti (9 farklı renk)
- State Machine'e uygun aksiyon butonları
- Detay butonu (modal ile ürün listesi)

Detay Modal

- Sipariş bilgi grid'i (ID, masa, durum, tarih, notlar)
- Ürün listesi (ad, adet, fiyat, opsiyonlar)
- Toplam tutar gösterimi
- AJAX ile dinamik yükleme

7.2 Progress Stepper — Yatay Durum Çubuğu

Dönüşüm: Önceki 5 adet stat kartı kaldırıldı. Yerine modern yatay stepper bar eklendi.

Tasarım: Her adım 36px daire (`.step-counter`) + bağlantı çizgileri (`::after` pseudo-element) ile birbirine bağlı.

Onaylandı Hazırlanıyor Hazır Teslim Edildi Ödendi (3) (2) (1) (0)

CSS Sınıfı	Görünüm	Kullanım
<code>.completed</code>	Yeşil dolgu + yeşil çizgi	Geçmiş adımlar (sipariş bu adımı aştı)
<code>.active</code>	Yeşil kenarlık + glow + scale(1.15)	Mevcut adım (siparişin bulunduğu durum)
<code>.cancelled</code>	Kırmızı dolgu	İptal edilmiş siparişlerde tüm adımlar
<code>.stat-pill</code>	Mini rozet (<code>min-width: 20px</code>)	Her adımın altında o durumdaki sipariş sayısı

```
// Sipariş satırına tıklandığında stepper'ı güncelle
function highlightProgress(row) {
  const durum = row.dataset.durum;
  const stepMap = {
    'Onaylandı': 1, 'Hazırlanıyor': 2,
    'Hazır': 3, 'TeslimEdildi': 4,
    'TamÖdendi': 5, 'KısmiÖdendi': 5
  }
```

```
});  
const step = stepMap[durum] || 0;  
// Tüm stepper-item'lara completed/active/cancelled class'ı ata  
document.querySelectorAll('.stepper-item').forEach((el, i) => {  
  el.classList.toggle('completed', i + 1 < step);  
  el.classList.toggle('active', i + 1 === step);  
  el.classList.toggle('cancelled', durum === 'İptal');  
});  
}
```

8. Müşteri Tarafı UX İyileştirmeleri

8.1 Sipariş Gönder — Profesyonel Onay Akışı

Önceki Durum: Browser `confirm()` ve `alert()` kullanılıyordu — profesyonel değildi.

Yeni Durum:

-  Bootstrap onay modali (Vazgeç / Gönder butonları)
- Spinner animasyonu (gönderim sırasında butona spinner eklenir)
- Başarı: Yeşil toast bildirimi + offcanvas'ta onay ekranı (SVG tik ikonu)
- Hata: Kırmızı toast bildirimi (3 saniye süreli)

8.2 Ayrı Sepet Sayfası Kaldırıldı

Değişiklik: `/sepet` rotasına giden kullanıcılar `/Menu/Index` 'e yönlendirilir. Offcanvas sepet paneli tek arayüz olarak kullanılır — daha modern ve tek sayfa deneyimi.

8.3 Menü Görsel Render Düzeltmesi

Hata: Admin panelinden ürüne eklenen görseller menü sayfasında görünmüyordu.

Neden: Template'de sabit `<div class="img-placeholder">` kullanılıyordu, `<img>` etiketi yoktu.

Çözüm: Koşullu render: `GorselUrl` doluysa `<img>`, boşsa gri placeholder gösterilir. CSS'e `.product-img { object-fit: cover }` stili eklendi.

9. SignalR — Gerçek Zamanlı Menü Güncellemesi

9.1 Mimari Genel Bakış

```
[Admin Paneli] | | Ürün/Kategori/Opsiyon CRUD işlemi | ▼ [AdminController] —  
SaveChangesAsync() → [PostgreSQL] | | _menuHub.Clients.All.SendAsync("MenuGuncellendi") | ▼  
[MenuHub] — WebSocket broadcast → [Tüm Müşteri Tarayıcıları] | | fetch('/menu/json') | ▼  
[DOM Güncelleme] Kategori tablaları + Ürün kartları yeniden çizilir
```

9.2 Backend Yapılandırması

Bileşen

Dosya

Açıklama

MenuHub	Hubs/MenuHub.cs	Lightweight SignalR Hub sınıfı — server→client broadcast
AddSignalR()	Program.cs	SignalR servislerinin DI container'a kaydı
MapHub<MenuHub>	Program.cs	Hub endpoint: /hubs/menu
IHubContext<MenuHub>	AdminController.cs	Constructor injection ile Hub context'e erişim
/menu/json	MenuController.cs	Tüm menü verisi JSON — kategoriler + ürünler + opsiyonlar

9.3 SignalR Notify Noktaları (11 adet)

Aksiyon	Metot	Sinyal
Kategori ekleme	KategoriEkle()	MenuGuncellendi
Kategori güncelleme	KategoriGuncelle()	MenuGuncellendi
Kategori silme	KategoriSil()	MenuGuncellendi
Ürün ekleme	UrunEkle()	MenuGuncellendi
Ürün güncelleme	UrunGuncelle()	MenuGuncellendi
Ürün silme	UrunSil()	MenuGuncellendi
Ürün pasife alma	UrunSil() (bağlı sipariş)	MenuGuncellendi
Ürün aktif/pasif	UrunToggle()	MenuGuncellendi
Opsiyon ekleme	OpsiyonEkle()	MenuGuncellendi
Opsiyon silme	OpsiyonSil()	MenuGuncellendi

9.4 Frontend SignalR Client

```
// CDN: microsoft-signalr 8.0.0
const menuConnection = new signalR.HubConnectionBuilder()
  .withUrl('/hubs/menu')
  .withAutomaticReconnect([0, 2000, 5000, 10000, 30000])
  .build();

menuConnection.on('MenuGuncellendi', async function() {
  const res = await fetch('/menu/json');
  const kategoriler = await res.json();
  yenidenCiz(kategoriler); // DOM'u tamamen yeniden oluştur
});

menuConnection.start(); // WebSocket bağlantısını başlat
```

yenidenCiz() Fonksiyonu:

- Kategori tablalarını yeniden oluşturur (tıklama event'leri dahil)

- Ürün kartlarını sıfırdan render eder (görsel, fiyat, "Tükendi" rozeti, opsiyonlar)
- IntersectionObserver'ı yeniden bağlar (scroll takibi)
- XSS koruması: `escapeHtml()` ile tüm kullanıcı verileri escape edilir

9.5 SignalR — Sipariş Bildirimi (SiparisGuncellendi)

[Müşteri Menüsü] | | Sipariş gönder (POST /siparis/olustur) | ▼ [SiparisController] — SaveChangesAsync() → [PostgreSQL] | | _menuHub.Clients.All.SendAsync("SiparisGuncellendi") | ▼ [MenuHub] — WebSocket broadcast → [Admin Siparişler Sayfası] | | fetch('/admin/siparisler-json') | ▼ [renderTable() + updateStats()] Tablo + stepper bar yeniden çizilir [bildirimSesiCal()] 🔊

Sinyal Noktası	Dosya	Tetiklenme Anı
SiparisGuncellendi	SiparisController.cs	Yeni sipariş oluşturulduğunda (Olustur endpoint'i)
SiparisGuncellendi	AdminController.cs	Sipariş durumu güncellendiğinde (AdminDurumGuncelle endpoint'i)

9.6 Admin Client — SignalR + Web Audio API Bildirim Sesi

```
// Admin Siparişler sayfası – SignalR client
const connection = new signalR.HubConnectionBuilder()
  .withUrl('/hubs/menu')
  .withAutomaticReconnect()
  .build();

connection.on('SiparisGuncellendi', async () => {
  const res = await fetch('/admin/siparisler-json');
  const { list } = await res.json();
  renderTable(list); // Tabloyu yeniden çiz
  updateStats(list); // Stepper bar sayılarını güncelle
  bildirimSesiCal(); // 🔊 Ding sesi çal
});
```

```
// Web Audio API – programatik bildirim sesi (harici dosya gereksiz)
function bildirimSesiCal() {
  const ctx = new (window.AudioContext || window.webkitAudioContext)();
  const osc = ctx.createOscillator();
  const gain = ctx.createGain();
  osc.connect(gain);
  gain.connect(ctx.destination);
  osc.type = 'sine';
  osc.frequency.setValueAtTime(830, ctx.currentTime);
  osc.frequency.setValueAtTime(1100, ctx.currentTime + 0.1);
  gain.gain.setValueAtTime(0.3, ctx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.01, ctx.currentTime + 0.5);
  osc.start(ctx.currentTime);
  osc.stop(ctx.currentTime + 0.5);
}
```

Web Audio API Avantajları:

- Harici ses dosyası gereksiz — tamamen programatik üretim (830Hz → 1100Hz sine wave)
- Base64 audio sorunları ortadan kalktı (mobilde oynatma engeli, dosya boyutu)
- Ses süresi: 0.5 saniye, exponential decay ile doğal "ding" efekti

10. Middleware Düzeltmeleri

10.1 AJAX-Aware 401 JSON Yanıtı

Hata: AJAX ile sipariş gönderildiğinde "Microsoft.EntityFrameworkCore..." is not valid JSON hatası alınıyordu.

Neden: TokenValidationMiddleware 401 yanıtında düz metin döndürüyordu. JavaScript `res.json()` parse edemiyordu.

Çözüm: `WriteUnauthorized()` helper metodu: AJAX isteklerini (Content-Type/Accept/X-Requested-With) tespit edip `{"success":false,"message":"..."}` JSON döndürür. Normal tarayıcı isteklerinde düz metin döner.

```
private static async Task WriteUnauthorized(HttpContext ctx, string msg)
{
    var isAjax = ctx.Request.Headers["Content-Type"]
        .ToString().Contains("application/json")
        || ctx.Request.Headers["Accept"].ToString().Contains("application/json")
        || ctx.Request.Headers["X-Requested-With"] == "XMLHttpRequest";

    ctx.Response.StatusCode = 401;
    if (isAjax) {
        ctx.Response.ContentType = "application/json";
        await ctx.Response.WriteAsync(
            $"{{\"success\":false,\"message\":\"{msg}\"}}");
    } else {
        await ctx.Response.WriteAsync(msg);
    }
}
```

10.2 SignalR Hub Yolu Exclude

`/hubs` yolu `ExcludedPaths` dizisine eklendi. SignalR WebSocket bağlantıları token doğrulamasından muaf — aksi halde hub bağlantısı başarısız olurdu.

11. Karşılaşılan Hatalar ve Çözümler

#	Hata	Kategori	Çözüm
---	------	----------	-------

1	InMemory Transaction Hatası <code>InvalidOperationException: Transactions not supported</code>	Testing	InMemory DB transaction desteklemediği için <code>ConfigureWarnings</code> ile <code>TransactionIgnoredWarning</code> bastırıldı + provider-aware service tasarımı.
2	RowVersion null Hatası (InMemory) <code>NullReferenceException: RowVersion is null</code>	Testing	InMemory provider <code>RowVersion</code> otomatik üretmiyor — default <code>null!</code> yerine <code>Array.Empty<byte>()</code> atandı.
3	ExecutionStrategy InMemory Uyumsuzluğu <code>InvalidOperationException: RelationalConnection required</code>	Architecture	<code>Database.ProviderName</code> kontrol edilerek runtime'da dallanma — PostgreSQL için <code>ExecutionStrategy</code> , InMemory için doğrudan çağrı.
4	AJAX 401 JSON Parse Hatası <code>"Microsoft.EntityFrameworkCore..." is not valid JSON</code>	Middleware	Konsol hatası: <code>SyntaxError: Unexpected token 'M'</code> . Middleware düz metin döndürüyordu, JS <code>res.json()</code> parse edemedi. <code>WriteUnauthorized()</code> helper ile AJAX tespiti ve JSON yanıt eklendi.
5	WriteUnauthorized Scope Hatası <code>CS0116: Namespace does not contain definition</code>	Build	Metot yanlışlıkla sınıf dışına yerleştirildi. <code>TokenValidationMiddleware</code> sınıfının kapanış parantezinden önceye taşındı.
6	Menü Görselleri Görünmüyor Admin'den yüklenen fotoğraflar menüde placeholder olarak kalıyordu.	UI/Template	Template'de <code><div class="img-placeholder"></code> vardı, <code></code> yoktu. Koşullu render: <code>GorselUrl</code> doluyorsa <code></code> , boşsa placeholder.
7	ICollection<UrunOpsiyon> Instantiation Hatası <code>CS0144: Cannot create instance of abstract type</code>	Build	<code>/menu/json</code> endpoint'inde <code>new()</code> yerine <code>new List<UrunOpsiyon>()</code> kullanıldı. <code>ICollection</code> arayüz olduğu için doğrudan <code>new()</code> ile örneklenemiyor.
8	DbUpdateConcurrencyException <code>Expected to affect 1 row(s), but actually affected 0 row(s)</code>	Runtime	Sipariş oluşturulurken RowVersion uyumsuzluğu. Transaction ve retry mekanizması zaten mevcut. Log kaydı: <code>[ERR] 2026-03-08 15:46:22</code>
9	Foreign Key Constraint İhlali <code>23503: FK_Oturumlar_Masalar_MasaId violated</code>	Runtime/DB	QR kod ile giriş yapılırken olmayan bir masa ID'si kullanıldı. Log kaydı: <code>[ERR] 2026-03-08 15:56:25</code> . Eşleşme doğrulaması kontrol edildi.
10	RowVersion PostgreSQL NOT NULL Hatası <code>23502: NOT NULL constraint failed on column "RowVersion"</code>	Runtime/DB	<code>IsRowVersion()</code> SQL Server'a özgüdür ve PostgreSQL'de otomatik değer üretmez. <code>IsConcurrencyToken().HasColumnType("bytea")</code> yapılandırmasına geçildi + <code>SiparisService</code> 'te her oluşturma/güncelleme öncesi <code>Guid.NewGuid().ToByteArray()</code> atandı.

dotnet process EXE Kildi

11

MSB3027: Could not copy ...
QRMenu.Web.exe ... being used by
another process

Build

Arka planda çalışan `dotnet run` prosesi EXE
dosyasını kilitledi. `taskkill /f /im dotnet.exe`
ile proses sonlandırıldı, build başarılı oldu.

12. Önemli Log Kayıtları (8 Mart 2026)

```
- Uygulama Başlatma -
2026-03-08 14:18:55 [INF] Veritabanı migration başarılı.
2026-03-08 14:18:55 [INF] QR Menü uygulaması başlatılıyor...

- İlk Bağlantı Sorunları -
2026-03-08 14:21:46 [WRN] Failed to determine the https port for redirect.
2026-03-08 14:21:46 [WRN] Token cookie bulunamadı. Path: /

- Sepet İşlemleri (Normal Akış) -
2026-03-08 14:28:42 [INF] Yeni sepet oluşturuldu. OturumId=17, SepetId=1
2026-03-08 14:38:09 [INF] Sepete yeni ürün eklendi. SepetId=1, UrunId=3, Adet=1
2026-03-08 14:38:18 [INF] Sepetten ürün çıkarıldı. DetayId=1

- QR Giriş -
2026-03-08 14:48:22 [INF] QR oluşturuldu. Masa=1
2026-03-08 16:42:10 [INF] QR giriş başarılı. MasaId=2, OturumId=25

- HATA: Concurrency Exception (Sipariş Oluşturma) -
2026-03-08 15:46:22 [ERR] An unhandled exception has occurred while executing the request.
  DbUpdateConcurrencyException: The database operation was expected to affect
  1 row(s), but actually affected 0 row(s); data may have been modified or
  deleted since entities were loaded.
  → at SiparisService.ExecuteSiparisOlusturAsync() :line 97

- HATA: Foreign Key İhlali (QR Giriş) -
2026-03-08 15:56:25 [ERR] Failed executing DbCommand (114ms)
  PostgresException (23503): insert or update on table "Oturumlar"
  violates foreign key constraint "FK_Oturumlar_Masalar_MasaId"
  → Geçersiz masa ID ile oturum oluşturma denemesi

- HATA: Sipariş Oluşturma DB Exception -
2026-03-08 16:26:03 [ERR] An unhandled exception has occurred while executing the request.
  → Sipariş detay kopyalama sırasında DB hatası

- Admin Ürün Güncelleme (Çoklu - Görsel Test) -
2026-03-08 16:38:04 [INF] Ürün güncellendi. Id=3
2026-03-08 16:38:22 [INF] Ürün güncellendi. Id=3
2026-03-08 16:38:32 [INF] Ürün güncellendi. Id=3
2026-03-08 16:38:43 [INF] Ürün güncellendi. Id=3
→ Görsel yükleme testi: 4 kez art arda güncelleme yapıldı

- Uygulama Yeniden Başlatma (SignalR Entegrasyonu Sonrası) -
2026-03-08 16:50:30 [INF] Veritabanı migration başarılı.
2026-03-08 16:50:31 [INF] Now listening on: http://localhost:5015
2026-03-08 16:50:41 [INF] Ürün güncellendi. Id=3
→ SignalR ile test: admin değişikliği → menüde anlık güncelleme
```

13. Test ve Doğrulama (59/59 PASSED)

Test Grubu	Test Sayısı	Durum
Ürün Servisi Testleri (Hafta 1)	7	✓ Geçti
Token/Oturum Testleri (Hafta 2)	9	✓ Geçti
Sepet ve Hesaplama Testleri (Hafta 3)	15	✓ Geçti
Sipariş Motoru Testleri (Hafta 4)	28	✓ Geçti

13.1 Sipariş Testleri Detayı (28 Test)

#	Test	Sonuç
Sipariş Oluşturma		
1	Başarılı sipariş oluşturma — sepet verisi doğru kopyalanmalı	✓
2	Boş sepet ile sipariş — hata fırlatmalı	✓
3	Geçersiz sepet ID — hata fırlatmalı	✓
4	Sipariş notları doğru kaydedilmeli	✓
State Machine — Geçerli Geçişler (10 Senaryo)		
5	Sepette → Onaylandı	✓
6	Onaylandı → Hazırlanıyor	✓
7	Onaylandı → İptal	✓
8	Hazırlanıyor → Hazır	✓
9	Hazırlanıyor → İptal	✓
10	Hazır → Teslim Edildi	✓
11	Teslim Edildi → Kısmi Ödendi	✓
12	Teslim Edildi → Tam Ödendi	✓
13	Kısmi Ödendi → Tam Ödendi	✓
14	Tam Ödendi → İade	✓
State Machine — Geçersiz Geçişler (6 Senaryo)		
15	Sepette → Hazırlanıyor (atladı!) — hata	✓
16	Onaylandı → Teslim Edildi — hata	✓
17	Hazır → Onaylandı (geri!) — hata	✓

18	İptal → Onaylandı (terminal!) — hata	✓
19	İade → Onaylandı (terminal!) — hata	✓
20	Olmayan sipariş ID — hata	✓
İptal & Sorgulama		
21	Onaylandı durumundan iptal	✓
22	Teslim Edildi durumundan iptal — hata (yalnız Onaylandı/Hazırlanıyor)	✓
23	Sipariş detayı getirme (detay satırları + ürün)	✓
24	Masa bazlı sipariş listesi (iptal hariç)	✓
25	Oturum bazlı sipariş listesi	✓
Geçerli Geçiş Kuralları		
26	Onaylandı'nın geçerli geçişleri: Hazırlanıyor, İptal	✓
27	İptal'in geçerli geçişleri: boş dizi (terminal)	✓
28	Teslim Edildi → İade geçişi	✓

```
> dotnet test tests\QRMenu.Tests\QRMenu.Tests.csproj
Başarılı! - Başarısız: 0, Başarılı: 59, Atlanan: 0, Toplam: 59 ✓
```

14. Bu Haftada Kullanılan Teknolojiler & Desenler

Teknoloji / Desen	Kullanım Amacı
State Machine Pattern	Sipariş yaşam döngüsü — kurala aykırı geçişleri engelleme
TransactionScope	Sepet → Sipariş dönüşümünü atomik yapma (ya hep ya hiç)
RowVersion / Timestamp	Eş zamanlı durum güncellemelerinde çakışma kontrolü (Optimistic Concurrency)
ExecutionStrategy	Npgsql retry logic ile transient hata toleransı
IFormFile + GUID	Güvenli fotoğraf upload (dosya adı sanitization + boyut/format kontrolü)
AJAX (Fetch API)	Sayfa yenilemeden kategori/ürün/opsiyon CRUD işlemleri
FormData	Fotoğraf ile birlikte form verisi gönderimi (multipart/form-data)
Bootstrap 5 Modals	3 farklı modal: Kategori düzenleme, Ürün ekleme/düzenleme, Opsiyon yönetimi
xUnit + InlineData/Theory	Parametrik test senaryoları ile 10 geçerli + 6 geçersiz state transition testi

ASP.NET Core SignalR	WebSocket tabanlı gerçek zamanlı iletişim — admin→müşteri menü güncellemeleri
IHubContext<T>	Controller'dan Hub dışında SignalR mesajı gönderme (DI ile injection)
AutomaticReconnect	SignalR client otomatik yeniden bağlanma (0s, 2s, 5s, 10s, 30s aralıklarla)
Bootstrap Toast	Profesyonel bildirimler — başarı (yeşil) ve hata (kırmızı) toast mesajları
IntersectionObserver	Scroll takibi ile aktif kategori tabı highlight — DOM yenilendikten sonra yeniden bağlanır
Web Audio API	Programatik bildirim sesi üretimi (830Hz→1100Hz sine wave, 0.5s decay) — harici dosya gereksiz
CSS Progress Stepper	Yatay adım çubuğu: daire (<code>.step-counter</code>) + bağlantı çizgileri (<code>::after</code>) + completed/active/cancelled state'leri
GitHub Actions CI/CD	Otomatik build → test → publish → SCP/SSH deploy pipeline. Secrets ile güvenli credential enjeksiyonu
variable-substitution	Microsoft action — appsettings.json içindeki placeholder'lara Secrets değerlerini enjekte eder

15. GitHub Actions — CI/CD Pipeline

15.1 Pipeline Akışı

```
[git push main] | ▼ [Checkout] → [.NET 8 Setup] → [Secrets Injection] | | variable-  
substitution@v1 | DB_USER, DB_PASSWORD, DB_SERVER → appsettings.json ▼ [dotnet test] | | 59/59  
pass ✅ ▼ [dotnet publish] → [ZIP] | | SCP → VDS ▼ [SSH] unzip + systemctl restart
```

15.2 Yapılandırma Detayları

Parametre	Değer	Açıklama
Tetikleme	<code>push → main</code>	Sadece main branch'e push yapıldığında çalışır
Runner	<code>ubuntu-latest</code>	GitHub-hosted Linux runner
.NET Sürüm	<code>8.0.x</code>	Proje ile aynı SDK sürümü
Publish Modu	<code>Release + self-contained</code>	Sunucuda .NET runtime gerektirmez
Deploy Yöntemi	SCP + SSH	appleboy/scp-action + appleboy/ssh-action

15.3 GitHub Secrets

Secret	Kullanım
<code>DB_USER</code>	PostgreSQL (Supabase) kullanıcı adı

DB_PASSWORD	PostgreSQL (Supabase) şifresi
DB_SERVER	PostgreSQL sunucu adresi (host:port/db)
VDS_HOST	Deploy hedef sunucu IP adresi
VDS_USERNAME	SSH kullanıcı adı
VDS_PASSWORD	SSH şifresi
VDS_PORT	SSH port numarası

Güvenlik: `appsettings.json` dosyası `.gitignore` ile repo dışında tutulur. Bağlantı bilgileri yalnızca GitHub Secrets üzerinden pipeline sırasında enjekte edilir — kod tabanında hiçbir credential bulunmaz.

16. 9 Mart Güncellemeleri — Bug Fix, Menü İçeriği & Opsiyon Düzenlemeleri

16.1 Mobil Admin Panel Düzeltmeleri

Sayfa	Sorun	Kök Neden	Çözüm
Admin Ürünler	Tablo fiyat kolonundan sonra kesiliyordu, işlem butonları görünmüyordu	<code>.urun-table-wrap</code> CSS'inde <code>overflow: hidden</code> base stilinde tanımlıydı	<code>overflow-x: auto; -webkit-overflow-scrolling: touch;</code> olarak güncellendi
Admin Siparişler	Tablo mobilde taşıyordu, yatay scroll çalışmıyordu	<code>.siparis-table-wrap</code> base CSS'inde overflow stili hiç yoktu, yalnız media query'de tanımlıydı	Base CSS'e <code>overflow-x: auto; -webkit-overflow-scrolling: touch;</code> eklendi, media query konsolide edildi

16.2 Ürün Silme Hatası Düzeltmesi (FK Restrict)

Hata: Kategorideki bir ürünü silmeye çalışınca "Bu kategoride ürün var, silinmiyor" uyarısı alınıyordu. Ürün silme butonu da çalışmıyordu.

Kök Neden: `SiparisDetay` ve `SepetDetay` tablolarında `DeleteBehavior.Restrict` FK tanımı mevcut. İptal edilmiş siparişlerin detay kayıtları fiziksel olarak hâlâ DB'de olduğu için ürün silinmiyor.

Çözüm: `UrunSil()` metodu güncellendi — ürün silmeden önce ilişkili `SepetDetaylar` ve `SiparisDetaylar` kayıtları `RemoveRange()` ile temizleniyor.

```
// AdminController.cs - UrunSil() güncelleme
var sepetDetaylar = await _context.SepetDetaylar
    .Where(sd => sd.UrunId == id).ToListAsync();
if (sepetDetaylar.Any()) _context.SepetDetaylar.RemoveRange(sepetDetaylar);

var siparisDetaylar = await _context.SiparisDetaylar
    .Where(sd => sd.UrunId == id).ToListAsync();
if (siparisDetaylar.Any()) _context.SiparisDetaylar.RemoveRange(siparisDetaylar);
```

```
_context.Urunler.Remove(urun);  
await _context.SaveChangesAsync();
```

16.3 Menü İçeriği Doldurma (Seed Data)

Yöntem: Mevcut Admin API endpoint'leri (/admin/kategori-ekle , /admin/urun-ekle , /admin/opsiyon-ekle) PowerShell scriptleri ile çağrılarak gerçek restoran menüsü oluşturuldu.

Kategori	Ürün Sayısı	Örnek Ürünler
Kahvaltı	3	Serpme Kahvaltı, Sahanda Yumurta, Menemen
Tostlar	3	Kaşarlı Tost, Karışık Tost, Tavuklu Tost
Burgerler	4	Klasik Burger, Cheese Burger, Tavuk Burger, Double Burger
Pizzalar	3	Margarita, Karışık Pizza, Tavuklu Pizza
Makarnalar	3	Bolonez, Alfredo, Pesto Makarna
Salatalar	3	Sezar Salata, Çoban Salata, Ton Balıklı Salata
Tatlılar	4	Çikolatalı Sufle, Frambuazlı Cheesecake, Künefe, Tiramisu
Kahveler	4	Espresso, Cappuccino, Mocha, Flat White
Sıcak İçecekler	4	Türk Kahvesi, Çay, Sahlep, Sıcak Çikolata
Soğuk İçecekler	4	Ice Latte, Ev Yapımı Limonata, Mango Smoothie, Buzlu Çay
Atıştırmalıklar	3	Patates Kızartması, Soğan Halkası, Tavuk Nugget

Toplam: 11 kategori, 38 ürün, 24 opsiyon (Boyut, Süt Tipi, Menü, Ekstra, Porsiyon, Sos grupları)

16.4 Opsiyon Grupları ve Yapılandırma

Grup	Ürünler	Seçenekler	Tip
Boyut	Burgerler, Pizzalar, Kahveler, Sıcak/Soğuk İçecekler	Küçük (0 TL), Orta (+10 TL), Büyük (+25 TL)	Radio (Zorunlu)
Menü	Burgerler	Menü — Patates+İçecek (+40 TL)	Radio (Zorunlu)
Süt Tipi	Kahveler	Normal Süt (0 TL), Yulaf Sütü (+5 TL), Badem Sütü (+5 TL)	Radio (Zorunlu)
Ekstra	Tostlar, Pizzalar	Ekstra Kaşar (+10 TL), Ekstra Sucuk (+15 TL), Ekstra Peynir (+15 TL)	Checkbox (İsteğe Bağlı)
Porsiyon	Atıştırmalıklar	Küçük Porsiyon (0 TL), Büyük Porsiyon (+15 TL)	Radio (Zorunlu)

16.5 Opsiyon Sıralama Düzeltmesi

Sorun: Opsiyonlar "Orta → Küçük → Büyük" şeklinde bozuk sırada gösteriliyordu (DB insertion order'a bağlıydı).

Çözüm: 3 farklı noktaya `.OrderBy(uo => uo.Opsiyon.EkFiyat)` eklendi:

- `MenuController.cs` — müşteri menü API'si (`/menu/json`)
- `AdminController.cs` — admin ürün detay (`/admin/urun-detay/{id}`)
- `Views/Menu/Index.cshtml` — Razor view (opsiyonJson serialization)

Sonuç: Küçük (0 TL) → Orta (+10 TL) → Büyük (+25 TL) doğru sıralama sağlandı.

16.6 Opsiyon Mantık Düzeltmeleri

Düzeltilme	Detay
Normal Süt (0 TL) eklendi	Süt Tipi grubunda sadece Yulaf (+5 TL) ve Badem (+5 TL) vardı — müşteriye bedava seçenek yoktu. Normal Süt (0 TL) tüm kahve ürünlerine eklendi.
Makarna opsiyonları kaldırıldı	Spagetti/Penne/Fettuccine seçenekleri gereksizdi — makarna ürünlerinden tüm opsiyonlar ve kullanılmayan opsiyon kayıtları temizlendi.
Sos grubu isteğe bağlı yapıldı	Sos grubu radio (zorunlu) olarak gösteriliyordu ancak müşteri sos istemeyebilir. JavaScript'te <code>isMulti</code> kontrolüne <code>'sos'</code> eklenerek checkbox (isteğe bağlı) yapıldı.

16.7 Türkçe Karakter Düzeltmesi

Sorun: PowerShell terminal encoding'i UTF-8 Türkçe karakterleri (ş, ç, ğ, ı, ö, ü, İ) bozuyordu. HTTP istekleriyle gönderilen "Kaşarlı" → "KaÅŸarlıÄ±" olarak veritabanına kaydedildi.

Çözüm: Geçici C# endpoint (`[HttpPost("/admin/fix-turkish")]`) oluşturuldu. C# source dosyaları UTF-8 encoding kullandığı için Türkçe karakterler doğru işlendi. 38 ürün + 12 kategori + 10 opsiyon güncellendi. Endpoint işlem sonrası koddan kaldırıldı.

17. 9 Mart — Karşılaşılan Hatalar ve Çözümler

#	Hata	Kategori	Çözüm
12	Mobil Admin Ürünler Tablosu Kesilmesi Fiyat kolonundan sonra içerik görünmüyor, butonlar erişilemiyor	CSS / Mobile	<code>.urun-table-wrap { overflow: hidden } → overflow-x: auto; -webkit-overflow-scrolling: touch;</code>
13	Mobil Admin Siparişler Tablosu Taşması	CSS / Mobile	Base CSS'e <code>.siparis-table-wrap { overflow-x: auto }</code> eklendi, media query konsolide edildi

Sayfa genişliğini aşiyor,
kaydırılamıyor

Ürün Silme Butonu

Çalışmıyor

14 23503: FK constraint
violated — Restrict FK
engeli

Database /
FK

SepetDetaylar ve SiparisDetaylar kayıtları
silme öncesi RemoveRange() ile temizleniyor

PowerShell UTF-8 Encoding Bozulması

15 "Kaşarlı" → "KaÄYarİÄ±" olarak
DB'ye kaydediliyor

Encoding /
i18n

PowerShell yerine C# geçici endpoint ile doğrudan DB
güncelleme — C# source UTF-8 BOM ile doğru
encoding

Opsiyon Sıralama Bozukluğu

16 Orta → Küçük → Büyük (DB
insertion order)

UX /
Sıralama

3 noktaya .OrderBy(uo => uo.Opsiyon.EkFiyat)
eklendi

Süt Tipi Grubunda Bedava

Seçenek Yok

17 Müşteri her durumda +5 TL
ödüyordu

UX / İş
Kuralı

"Normal Süt" (0 TL) opsiyonu oluşturulup tüm kahve
ürünlerine eklendi

Sos Grubu Yanlış Zorunlu

Gösterimi

18 Müşteri sos istemese bile
seçim yapması gerekiyordu

UX / İş
Kuralı

JS'te isMulti kontrolüne 'sos' eklenerek
checkbox (isteğe bağlı) yapıldı

18. 10 Mart Güncellemeleri — Opsiyon Zorunlu Toggle, Garson Çağır, Fiş Overlay, UX & Bug Fix

18.1 Dosya Değişiklikleri (10 Mart)

```
src/  
├── QRMenu.Core/Entities/  
│   └── Opsiyon.cs ← Zorunlu (bool, default=true) alanı eklendi  
├── QRMenu.Data/Migrations/  
│   ├── ...OpsiyonZorunluAlani.cs ← [YENİ] Opsiyonlar tablosuna Zorunlu kolonu  
│   └── ...UrunAciklamaGuncelleme.cs ← [YENİ] 47 ürün açıklama UPDATE SQL  
├── QRMenu.Web/  
│   ├── Controllers/  
│   │   ├── AdminController.cs ← Opsiyon fiyat+zorunlu güncelleme, UTC→TR fix  
│   │   ├── MenuController.cs ← JSON'da zorunlu alanı  
│   │   └── SiparisController.cs ← GarsonCagir endpoint + UTC→TR fix  
│   ├── ViewModels/  
│   │   └── UrunYonetimViewModel.cs ← OpsiyonFormViewModel'e Zorunlu eklendi  
│   ├── Views/  
│   │   ├── Admin/  
│   │   │   ├── Urunler.cshtml ← Zorunlu switch toggle + badge  
│   │   │   └── Siparisler.cshtml ← Garson overlay+alarm, Fiş overlay (modal kaldırıldı)  
│   │   ├── Menu/  
│   │   │   └── Index.cshtml ← Zorunlu logic, bottom bar, tekrar sipariş, emoji placeholder  
│   │   └── Sepet/
```

```
// Migration: OpsiyonZorunluAlani
migrationBuilder.AddColumn<bool>(
    name: "Zorunlu",
    table: "Opsiyonlar",
    type: "boolean",
```

```
nullable: false,  
defaultValue: true); ← Mevcut opsiyonlar zorunlu olarak işaretlenir
```

Admin UI

⚙️ Form Değişiklikleri

- Switch toggle: "Zorunlu" (açık = zorunlu, kapalı = opsiyonel)
- Opsiyon listesinde `Zorunlu` / `Opsiyonel` badge'i
- OpsiyonEkle endpoint Zorunlu parametresini alıyor
- Mevcut opsiyon güncellendiğinde Zorunlu değeri de güncelleniyor

📱 Müşteri Tarafı Davranış

- **Zorunlu grup:** Radio button — bir seçim zorunlu
- **Opsiyonel grup:** Checkbox — hiçbirini seçilmeyebilir
- Grup başlığında `(Zorunlu)` veya `(Opsiyonel)` etiketi
- "Sepete Ekle" butonu tüm zorunlu gruplar seçilene kadar disabled

18.4 Garson Çağır Sistemi (SignalR + Rate Limiting)

```
[Müşteri Menüsü] | | "Garson Çağır" 📞 butonuna tıkla | ▼ [POST /siparis/garson-cagir] | | Rate  
Limit: 3 çağrı / 5 dakika | ▼ _menuHub.Clients.All.SendAsync("GarsonCagrisi", masaNo) | ▼  
[Admin Siparişler Sayfası] | | → 📞 Tam ekran overlay - "Masa X Garson Çağırıyor!" | → 🔊 3  
saniye alarm sesi (880Hz/660Hz bip bip, Web Audio API) | → ❌ Tıklayınca kapanır
```

Bileşen	Detay
Endpoint	POST /siparis/garson-cagir — [EnableRateLimiting("GarsonCagirPolicy")]
Rate Limit	FixedWindowRateLimiter: 3 istek / 5 dakika pencere, IP bazlı
SignalR Event	GarsonCagrisi(masaNo) — tüm bağlı istemcilere broadcast
Admin Overlay	Siyah arka plan + beyaz kutu, pulse animasyon, 📞 emoji, masa numarası
Alarm Sesi	Web Audio API: 880Hz→660Hz, 0.2s aralıkla 6 tekrar = 3 saniye alarm

18.5 Sipariş Fiş (Receipt) Overlay

❌ Eski Tasarım (Modal)

- Bootstrap modal kutusu içinde küçük fiş
- Barista için okunması zordu
- Standart `.modal-dialog` genişliğinde

✅ Yeni Tasarım (Overlay)

- Tam ekran karanlık overlay + beyaz fiş kartı
- 480px genişlik, Courier New font
- Başlık: 2rem, Ürünler: 1.15rem, Toplam: 1.6rem
- Overlay arka planına veya × butonuna tıklayınca kapanır
- Kesikli çizgiler ile gerçek fiş görünümü

18.6 Tekrar Sipariş Gönderme (Çok Kullanıcı Masa Desteği)

Senaryo: Masada 2–3 arkadaş farklı cihazlardan sipariş verebilir. Sipariş gönderdikten sonra tekrar sipariş vermek isteyebilirler.

Eski Davranış: Sipariş sonrası sepet "Siparişiniz Alındı!" ekranına kilitleniyor, buton "Sipariş Gönderildi ✓" olarak deactive oluyordu.

Yeni Davranış:

- Toast ile "Siparişiniz mutfağa iletildi!" bildirimi gösterilir
- Sepet içeriği `fetchCartDetails()` ile yenilenir (boş sepet görünür)
- Buton tekrar "Siparişi Gönder" olarak aktif hale gelir
- Ayri sepet sayfasında (`/sepet`) `location.reload()` ile temiz sayfa

18.7 Ürün Açıklamaları & Görsel İyileştirmeleri

Açıklama Güncelleme (47 Ürün)

Yöntem: EF Core migration (`UrunAciklamaGuncelleme`) içinde SQL UPDATE ifadeleri ile tüm ürünlerin Türkçe ve İngilizce açıklamaları güncellendi.

Stil: Kısa bilgi yerine çekici, detaylı açıklamalar. İçerik, hazırlanış yöntemi ve servis bilgisi.

Ürün	Eski Açıklama	Yeni Açıklama
Americano	Espresso ve sıcak su	Yoğun espresso ve sıcak su ile hazırlanan americano. Sade kahve sevenler için güçlü ve aromatik bir tercih.
Çikolatalı Sufle	Sıcak çikolatalı sufle, vanilya dondurması ile	Sıcak sıcak servis edilen, içi akışkan çikolatalı sufle. Yanında vanilya dondurması ile unutulmaz bir deneyim.
Klasik Burger	150gr dana köfte, marul, domates, turşu	150gr el yapımı dana köfte, taze marul, domates, turşu ve özel burger sosu ile brioche ekmeğinde.
Brownie	Çikolatalı brownie	Yoğun çikolatalı, ıslak kıvamda ev yapımı brownie. Yanında dondurma ile mükemmel.

Kategori Emoji Placeholder

Görseli yüklenmemiş ürünler için gri kutu yerine kategoriye uygun emoji gösterilir. CSS `::after` pseudo-element ve `data-kategori` attribute selector ile implementasyon.

Kategori	Emoji	CSS Selector
Sıcak İçecekler / Kahveler	☕	<code>.img-placeholder::after</code> (varsayılan)
Soğuk İçecekler	🧊	<code>[data-kategori*="Soğuk"]::after</code>
Kahvaltı	🍳	<code>[data-kategori*="Kahvaltı"]::after</code>
Tostlar	🥪	<code>[data-kategori*="Tost"]::after</code>
Burgerler	🍔	<code>[data-kategori*="Burger"]::after</code>
Pizzalar	🍕	<code>[data-kategori*="Pizza"]::after</code>

Makarnalar		[data-kategori*="Makarna"]::after
Salatalar		[data-kategori*="Salata"]::after
Tatlılar		[data-kategori*="Tatlı"]::after
Atıştırmalıklar		[data-kategori*="Atıştırmalık"]::after

Açıklama Satır Limiti

Ürün kartlarında açıklama alanı `-webkit-line-clamp: 2` → `3` olarak genişletildi. Daha detaylı açıklamalar kırpılmadan okunabilir.

18.8 Sipariş Saati UTC Fix (v1 → v2)

Sorun (v1 — 10 Mart): Sipariş oluşturma saati DB'de `DateTime.UtcNow` ile kaydediliyor (doğru). Ancak JSON endpoint'lerde `.ToLocalTime()` çağrılmadığı için fiş ve tabloda saat 3 saat geri yazılıyordu (UTC+0 yerine UTC+3 Türkiye).

İlk Çözüm (v1): `.ToLocalTime()` eklendi — *localhost'ta çalışıyor çünkü makine TZ = Europe/Istanbul.*

Sorun (v2 — 11 Mart): `.ToLocalTime()` sunucu makinesinin zaman dilimine bağlı. Canlıda (VDS/Docker) sunucu UTC saat dilimindeyse yine 3 saat geri gösterilir. Nitekim deploy sonrası admin panelde sipariş listesi 09:54, fiş 12:54 gösterdi → Razor tarafı hâlâ `.ToLocalTime()` kullanıyordu.

Etkilenen Noktalar:

- `AdminController` → `/admin/siparis-detay/{id}` (fiş tarihi)
- `AdminController` → `/admin/siparisler-json` (tablo tarihi)
- `SiparisController` → `/siparis/{id}` (müşteri detayı)
- `SiparisController` → `/siparis/siparislerim` (müşteri listesi)
- `Siparisler.cshtml` → Razor view'daki sipariş tarihi kolonu

Kesin Çözüm (v2): `.ToLocalTime()` tamamen kaldırıldı. Yerine açık saat dilimi dönüşümü getirildi:

```
private static readonly TimeZoneInfo _turkeyTz =
    TimeZoneInfo.FindSystemTimeZoneById(
        OperatingSystem.IsWindows()
            ? "Turkey Standard Time"
            : "Europe/Istanbul");

private static string ToTurkeyTime(DateTime utc) =>
    TimeZoneInfo.ConvertTimeFromUtc(
        DateTime.SpecifyKind(utc, DateTimeKind.Utc), _turkeyTz)
        .ToString("dd.MM.yyyy HH:mm");
```

Böylece sunucu hangi TZ'de olursa olsun (UTC, CET, vb.) her zaman Türkiye saati gösterilir.

`OperatingSystem.IsWindows()` ile Windows/Linux farkı da ele alındı.

19. 10 Mart — Karşılaşılan Hatalar ve Çözümler

#	Hata	Kategori	Çözüm
19	Opsiyon Fiyat Reuse Bug "Sade" opsiyonu "Şeker" grubunda +1.00 TL gösteriyordu, 0 TL olmalıydı	İş Kuralı / Backend	OpsiyonEkle endpoint'i mevcut opsiyon bulunduğunda EkFiyat'ı güncellemiyor, eski değeri kullanıyordu. Farklıysa <code>SaveChangesAsync()</code> ile güncelleme eklendi.
20	Frontend Zorunlu Alanı Eksik Tüm opsiyon grupları "Opsiyonel" (checkbox) olarak gösteriliyordu	Frontend / Razor	Razor inline serialization'da <code>zorunlu =</code> <code>uo.Opsiyon.Zorunlu</code> eksikti. Eklendi + JS'de <code>opsList[0].zorunlu === true</code> strict equality kontrolü.
21	Sipariş Saati 3 Saat Geri (v1) Fiş ve tabloda 23:00 yerine 20:00 yazıyordu	Backend / Timezone	İlk çözüm: 4 JSON endpoint'te <code>.ToLocalTime()</code> eklendi. Localhost'ta çalıştı fakat sunucuda TZ bağımlılığı kaldı (bkz. #26).
22	Sipariş Butonu Kilitleniyor "Sipariş Gönderildi ✓" sonrası yeniden sipariş verilemiyor	UX / Frontend	Sepet içeriği başarı ekranına dönüşüyordu. Yerine <code>fetchCartDetails()</code> + buton reset + toast bildirimi konuldu.
23	Fiş Barista İçin Küçük Bootstrap modal box içindeki fiş okunması zordu	UX / Tasarım	Modal tamamen kaldırıldı. Tam ekran overlay + büyük fontlar (başlık 2rem, ürün 1.15rem, toplam 1.6rem) ile değiştirildi.
24	Opsiyon Fiyat Input Ondalık Sorunu Küsuratlı fiyat girildiğinde input kabul etmiyordu	Frontend / HTML	Input <code>step</code> attribute'u <code>"any"</code> olarak güncellendi.
25	dotnet build EXE Kilidi <code>MSB3027: Could not copy QRMMenu.Web.exe</code>	Build / Process	<code>taskkill /IM QRMMenu.Web.exe /F</code> ile arka plan prosesi sonlandırıldı.

19.5. 11 Mart — Karşılaşılan Hatalar ve Çözümler

#	Hata	Kategori	Çözüm
26	Sunucuda Sipariş Saati Hâlâ Yanlış <code>.ToLocalTime()</code> sunucu UTC'deyse çalışmıyor, canlı ortamda 3 saat geri	Backend / Timezone	<code>ToLocalTime()</code> kaldırıldı → <code>TimeZoneInfo.ConvertTimeFromUtc()</code> ile açık "Europe/Istanbul" dönüşümü. <code>OperatingSystem.IsWindows()</code> ile Windows/Linux TZ ID farkı yönetildi. AdminController + SiparisController'da static helper.
27	Razor View'da ToLocalTime Kalmış Admin sipariş listesi (Siparisler.cshtml) hâlâ <code>@s.OlusturmaTarihi.ToLocalTime()</code>	Frontend / Razor	Razor <code>@{ }</code> bloğuna <code>TimeZoneInfo</code> + <code>Func<DateTime,string> toTR</code> lambda eklendi. <code>@toTR(s.OlusturmaTarihi)</code> ile değiştirildi.

20. Gelecek Hafta Planı (Hafta 5) — Durum

Hafta 5 — Mutfak & Garson Paneli + Ödeme Sistemi + Deploy:

Görev	Durum	Açıklama
GitHub Actions CI/CD Pipeline	✅ Tamamlandı	<code>.github/workflows/deploy.yml</code> → checkout → .NET 8 kurulum → appsettings inject → test → publish → SCP → SSH restart
GitHub Secrets Yapılandırması	✅ Tamamlandı	<code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_SERVER</code> , <code>VDS_HOST</code> , <code>VDS_USERNAME</code> , <code>VDS_PASSWORD</code> , <code>VDS_PORT</code> secrets tanımlı
SignalR Altyapısı (mutfak/garson)	⚠️ Kısmen	<code>MenuHub</code> aktif: <code>SiparisGuncellendi</code> + <code>GarsonCagrisi</code> event'leri broadcast ediliyor. Ayrı mutfak/garson ekranına bildirim henüz yok.
Ödeme Akışı (backend)	⚠️ Kısmen	<code>SiparisDurum</code> enum'unda <code>TeslimEdildi=4</code> , <code>KismiOdendi=5</code> , <code>TamOdendi=6</code> tanımlı. State machine + <code>Odeme</code> entity mevcut. Ödeme UI/formu henüz yok.
Mutfak Ekranı (Kanban)	🔧 Geliştirilecek	Onaylandı → Hazırlanıyor → Hazır kartları, gerçek zamanlı güncelleme
Garson Paneli	🔧 Geliştirilecek	Masa bazlı sipariş listesi + durum güncelleme arayüzü
Admin Dashboard İstatistikleri	🔧 Geliştirilecek	Günlük/haftalık sipariş özeti, gelir grafikleri, popüler ürünler